International Journal of Computers and Applications

ISSN: 1206-212X (Print) 1925-7074 (Online) Journal homepage: www.tandfonline.com/journals/tjca20

A biological-like synthesis framework for software engineering environments

Peraphon Sophatsathit

To cite this article: Peraphon Sophatsathit (2024) A biological-like synthesis framework for software engineering environments, International Journal of Computers and Applications, 46:4, 208-217, DOI: 10.1080/1206212X.2023.2301183

To link to this article: https://doi.org/10.1080/1206212X.2023.2301183

Published online: 09 Jan 2024.

INTERNATIONAL JOURNAL OF COMPUTERS AND APPLICATIONS
2024, VOL. 46, NO. 4, 208–217
https://doi.org/10.1080/1206212X.2023.2301183

A biological-like synthesis framework for software engineering environments

Peraphon Sophatsathit

Advanced Virtual and Intelligent Computer Center (AVIC), Department of Mathematics and Computer Science, Faculty of Science, Chulalongkorn University, Bangkok, Thailand

ABSTRACT

This research proposes a Biological-like Architecture for Software Systems (BASS) that make up of software components. The design principle is to mimic the simplicity of uni-cellular life form as fixed-sized blocks linearly arranged as a flat array so that all references to those components can be reached directly. Each component holds its attributes and operations to permit self-execution without external support. Like living cells, this autonomy is modeled to be short-lived that undergoes a three-stage life cycle, namely, *creation*, *sustainment*, and *cessation*. This cycle must run to completion within a predefined Time-To-Live (TTL) limit. When the time limit of an existing component expires, a new component is created to replace *in situ* to conserve system energy and overhead. A simulation is conducted to test the operating characteristics of this three-stage life cycle under the time limit. The results reveal that component replacement *in situ* is practically viable. It is envisioned that BASS will offer an energy-efficient environment that can alleviate software systems resource utilization considerably.

ARTICLE HISTORY
Received 5 May 2020
Accepted 7 May 2021

KEYWORDS
Fixed size block; creation; sustainment; cessation; replacement *in situ*

1. Introduction

Traditional software architecture presets many constituent components that possess a number of design attributes. For example, an architectural construct of a utility set could be a temporary or permanent attribute, while the functional characteristics might be specifically derived from some basic requirements. An operating system (OS) is a comprehensive case to echo the above argument. The temporary components are numerous, namely, caches, buffers, processes, pipes and filters, etc., whereas the permanent ones are files, directories, bootstrap programs, devices, etc. The functional characteristics can range from general utilities such as shell commands, editor, to special programs such as text formatting programs, network APIs, device drivers, etc. Fowler outlines software architecture as follows.

> There are two common elements: One is the highest-level breakdown of a system into its parts; the other, decisions that are hard to change. It's also increasingly realized that there isn't just one way to state a system's architecture; rather, there are multiple architectures in a system, and the view of what is architecturally significant is one that can change over a system's lifetime [1].

The above arguments represent man-made complex artifacts that are theoretically well-structured, configurable, and can run indefinitely. Unfortunately, one of the shortfalls of this construct is its limitations induced by its own creating principles such as time and space complexities. This study looks into the basic building blocks of software architecture if there are alternatives to build software systems in a way mimicking natural artifacts, thereby the theory-ridden design process can be lessened.

From the oriental belief, the human body is made up of four substances, namely, earth, water, air, and fire. What makes it work is the soul that controls this living contraption. The hardware and software systems are analogous to the body as depicted in Figure 1.

The intricacy of architectural relationships to be drawn from this analogy begins at the highest level of body abstraction. The basic five senses, namely, sight, smell, touch, taste, and hearing, functioned by the input organs are the natural marvel that no artificially man-made devices can substitute. An even more intricacy is their integrated operations that consolidate all forms of input senses for instantaneous processing by the brain, subconscious mind, and instinct. The result is then interpreted, responded, or acted on accordingly. All these activities are completed in splits of a second. Further research deep into the structural construct of these organs unveils one common building block, i.e. the Deoxyribonucleic Acid (DNA) which is fundamentally composed of Adenine (A), Thymine (T), Cytosine (C), and Guanine (G) nucleotides. The functional aspect of each organ has long been discovered and established. From Figure 1, hardware and software exhibit *simple* design principles that map directly from 0 and 1 to AND, OR, NOT gates and software type hierarchy (see Figure 3). No qubit and superposition of quantum computing [2] or full library support of DNA computing [3] constructs to complicate the structure, function, and behavior of the proposed system. However, [4] proposed one good DNA computing prospect that could encode the Hamiltonian path to solve it in a single molecule. Unfortunately, such a powerful encoding scheme was relatively complicated to adopt since it would defeat the 'simple' design philosophy of the proposed work. This will be elucidated subsequently.

In order to uphold hardware 0 and 1 simplicity, software should be architected to mimic its counterpart. One predominant example is the uni-cellular life form which autonomously lives, reproduces, and dies. It in turn can combine to form more complicated multi-cellular life forms that work together seamlessly as a single living creature. If we could model software in the same manner, software would be able to work with its hardware counterpart seamlessly. It should not be superimposed on hardware as an afterthought. Thus, the objective of study is to propose a Biological-like Architecture for Software Systems (BASS) that will attain the following four characterizations: (1) simple and straightforward linear structure and algorithm design

CONTACT Peraphon Sophatsathit sperapon@gmail.com Advanced Virtual and Intelligent Computer Center (AVIC), Department of Mathematics and Computer Science, Faculty of Science, Chulalongkorn University, Bangkok 10330, Thailand

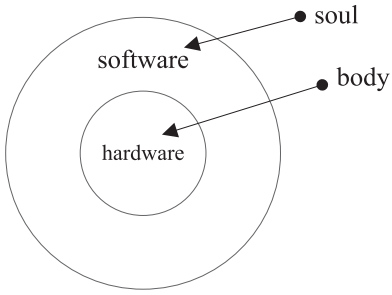

Figure 1. The human body vs the computer system.

for direct technology transfer to hardware, (2) fixed size to entail fast access and retrieval, (3) simple *in situ* replacement for space efficiency, and (4) no complex theoretical imposition.

This paper is organized as follows. Section 2 recounts some relevant researches that will be further exploited. Section 3 methodically describes two architectural propositions of BASS in three aspects, namely, structure, function, and behavior. A concise verification of both propositions is also presented. Section 4 explains the simulation experiment. Section 5 concludes a few novelty aspects of BASS with some final thoughts.

2. Related work

Race and ethnicity have long been used but were controversial and misunderstood of human classification [5]. was critical of the use of races [6]. explained the multi-dimensional of genetic variations by genomic resources in human populations. Whatever the classifications might be, biological heterogeneity offers a range of human genetic compositions. Such constructs, complicated as they appear, are made up of the aforementioned DNA to carry the genetic code of living cells. This notion, though has not yet been adopted in software systems, can be drawn as an analogy to software architecture that has varieties of patterns composing the software systems.

One of the well-established architectural constructs is modularization. Parnas [7] discussed related criteria using Keyword in Context (KWIC) as an example that consisted of (1) the task which was to build a contextualized index for the text, (2) input which was a set of lines of text, and (3) Output which was the set of all circular shifts of all lines in alphabetical order. These in turn entailed several flexible software architectural designs such as function-based, action-based, pipe and filter, event-condition-action, and implicit invocation.

Meierk [8] explained some key architectural designs, for instance, built to change instead of building to last models to analyze and reduce risks, used model and visualization, as a communication and collaboration tool, and identified key engineering decisions. Both suggestions pointed to flexible design considerations and decisions for a well-planned system architecture. Research endeavors on Software Engineering Environment (SEE) support [9] attempted to establish a framework of reference model (RM) for architectural standardization process, encompassing platform suppliers, environment suppliers, tool suppliers, and users. The underlying RM provided service groups as follows: (1) object management services, such as metadata, storage, persistence, archive, backup, relationship, name, distribution, location, replication, transaction, concurrency, synchronization, process support, access control, common schema, composite object, data interchange; (2) process management services, such as development, enactment, resource, monitoring, transaction; (3) communication services, such as data sharing, inter-process communication, network, message, event notification; (4) operating system services, such as synchronization, input and output, file storage, memory management; (5) user interface services, such as metadata, session, dialog, presentation, security, internationalization; (6) policy enforcement, such as identification and authentication, mandatory and discretionary access control, audit; and (7) framework administration services, such as registration, metrication, sub-environment, license management.

Abraham [10] and Xia [11] introduced coordination schemes using middleware and Secure InterlinK Architecture (SIKA), respectively, which could serve as a computation exchange for various BASS component operations.

In order to support direct software to hardware technology transfer, all software architectural differences, design, data collected must boil down to 0 and 1 to conform with the underlying hardware, e.g. Big/Little-endian architectures. BASS is set out to simplify these hardware constructs that support the software system operations, in particular, memory management systems [12]. proposed the process and memory integration to solve the memory wall problem [13]. introduced a memory access scheduler consisting of a comprehensive memory access scheduling architecture that supposedly alleviated memory bandwidth bottleneck. Current research and development efforts have established various techniques to efficiently handle such outgrown storage problems, leading to contention for shared resources on distributed environments such as non-uniform memory access (NUMA) [14]. However, the insatiable demands for massive storage do not dwindle down, cluttering the memory pool, and demanding more computational power of multiple tightly-coupled computer systems [15]. simulated the biological-like memory architecture using modified First-In-First-Out (FIFO) scheme and Time-To-Live (TTL) as a limiting factor to gauge how the scheme stacked up against well-established memory allocation algorithms such as pure FIFO, shortest remaining time first (SRTF), and round-robin (RR). The prospect proved to out-pace all schemes except pure FIFO due to context switch problem. Moreover, the modified FIFO scheme was starvation free and suitable for hardware implementable without any supporting complex allocation algorithms as oppose to other comparable schemes.

Recent developments of DNA computing [4,16,17] shed some lights to the adoption of biological approach for solving classical computing problems such as NP problems. A number of intrinsically complex problems that could not be efficiently solved by conventional electronic computers were handled by DNA computing approach [417] exploited the sequential code of 0's and 1's data in the computer to be adapted on the DNA computing storage scheme. The revolutionary hindsight instills more research endeavors on biological adaptation of efficient and effective computing capabilities. All these prior works constitute the hypotheses for the design of BASS. They are:

H1) The component construct adheres to uni-cellular structure, that is, autonomy;

H2) This construct is linearly modeled after nature cellular structure, that is, nucleotide, codon, chromosome and cell;

H3) Every component splits to create new ones (creation), undergoes some activities (sustainment), and finally goes away (cessation);

H4) Formation of a component is regulated by H3 to be linearly attached (created) or detached (go away) to the above construct;

H5) Each component lasts only TTL duration just as a uni-cellular life form has limited short life span;

H6) The basic types are the mandatory composition of every component; and

H7) Since nature is simple, so should BASS. No complex structures and algorithms are adopted.

In order to satisfy the above objectives of the study, some questions on how to accomplish the four characterizations of BASS arise:

Q1) What are the design principles to map such simple and straightforward linear structure and algorithm to hardware?

Q2) How can space be efficiently handled in ways that enhance access and retrieval speed?

Q3) Why doesn't BASS incorporate DNA and quantum computing to handle complex computation problems?

3. Proposed reference architecture

This study proposes a reference architecture of SEE characterization that mimics the simplicity of nature artifact, i.e. the human body. Two propositions are established based on the observation from Figure 1 as follows:

Proposition 1: There are simple building blocks on which software components can be built to constitute the software systems.

Proposition 2: System components must maintain a direct reference to the basic building blocks.

These propositions convey the basic construct of BASS. From the human analogy, the body is made of smaller organs which in turn are made of cells. The basic building blocks of these cells are nucleotides. A uni-cellular life form demonstrates how a single natural building block can survive and grow. By the same token, the basic building blocks of BASS are file blocks that make up a component. A collection of components make up BASS as shown in Figure 2. The design principle is to keep all components as simple as the uni-cellular life form by arranging components in a linear structure that permits fast, easy access and disposal just like cells.

To simply put, BASS consists of linearly arranged (similar to a one-dimension array) independent components that are detachable like human skin cells. Each component is autonomous and made of basic types (or ATCG in cell). It is created to perform some tasks within the limit of TTL (which can be renewed) and goes away. This life span is modeled after skin cells where an old (dying) cell comes off (or is detached from BASS) and the new ones are created (or attached to BASS) as replacement. Components (cells) form component group (organs) and eventually BASS (body).

The propositions are elucidated descriptively to follow the above forerunners, namely, modular design [7], data and tools abstraction, software engineering environments and reference model integration [18], uniqueness of DNA strands, and simple FIFO discipline. Details on how software components and their environments are laid out will be described in three aspects, namely, structure, function, and behavior.

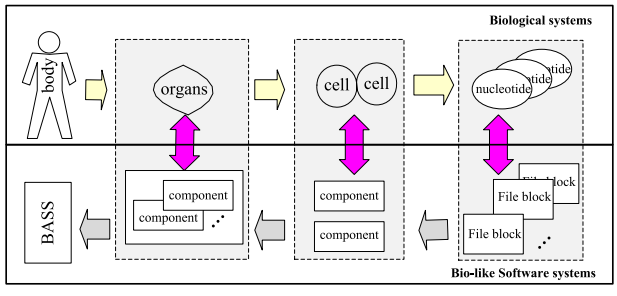

Figure 2. The proposed software systems architecture.

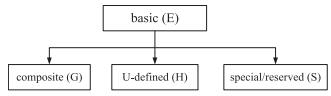

Figure 3. File type hierarchy.

3.1. Structural aspect of system architecture

As evident abound in archeology, many creatures were extinct while many evolved over the years. Conjectures on what the world centuries and millennia from now would look like have been attempted. One proposition that stood out from such predictions was the survival of cockroaches [19]. The fact that they are biologically simple renders them survive all adversaries as they could adapt to endure various environmental transformations. By the same token, the structural aspect of hardware and software can mimic such endurance by adhering to their simple building blocks, i.e. gates in hardware and bits in software. As such, they can maintain their close relationship to each other. Unfortunately, the intermediate and high-level constructs fade in and out as computer system architecture evolves, while their resemblance begins to divert from each other. For example, vacuum tubes, transistors, on the hardware side, and APL programming, HIPO chart, on the software side, became rarities and were replaced by integrated circuits and high-level/natural languages or software design patterns.

The proposed BASS scheme intends to preserve the simplicity of software architecture and keep it in close relationship with hardware counterpart. As a consequence, the hierarchy of software abstractions is reduced by establishing basic premises that govern the configuration of the proposed architecture as follows:

(1) Artifacts are countably finite and take the form of a component which represents the system construct of stored objects.
(2) All sizes of component substructures are fixed but configurable to permit maximal provision for hardware-level implementation.

Note that the above premises emphasize on fixed size structure to support performance efficiency. This size, in essence, is configurable to suit the underlying operating platform. This will be apparent in the design of BASS file systems in the experiment section (Figures 7 and 8).

In order for the proposed architecture to support the above propositions, it is essential that design of the software systems be compatible with their environments. The basis of BASS is file block that is characterized by its typing which is organized as shown in Figure 3. This hierarchical typing hierarchy is described as shown below.

- basic types (**E**) such as int (A), binary (B), char (C), note (D) that denotes sound attributes such as intensity and frequency;
- composite type (**G**) which denotes combinations of the basic types;
- user defined type (**H**); and
- special/reserved type (**S**).

The difference between composite and user-defined types is that the former builds a new component by combining several basic types, while the latter could be built from extending existing basic types or a new type defined by the user. For example, a composite type component could be composed of int (A), binary (B), and char (C). A U-defined component could be composed of extended binary type with a create-your-own method to manipulate the component or a new U-defined type for each 5-sense component representation.

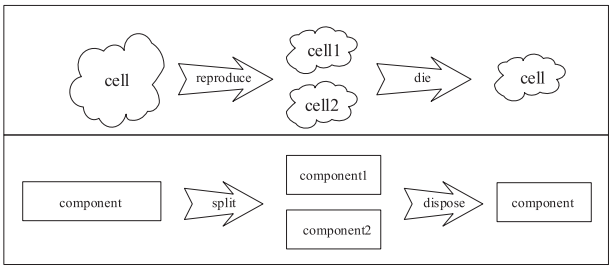

Figure 4. Cell (component) creates (splits), sustains, and ceases (dies).

Table 1. Summary of type description.

Type	OpCode	Operation
	E	
int (A)	ADD, SUB	Add, subtract
binary (B)	AND, OR, NOT, COM, SHT, XOR, BDD	Bitwise and, or, not, complement, shift, exclusive-OR, binary add
char (C)	ORD	Lexical order
note (D)	ITS, FRQ	Intensity, frequency
	G	
A,B,C,D	–	Composition of type **E**
	H	
U-defined	–	User define type
	S	
Reserved	–	–

The above setup only takes care of sight (text and image) and sound (audio) contents of file representations. The remaining three senses are more involved and complicated to represent. For example, touch includes texture, temperature, hardness; smell or scent includes odor, temperature; and taste includes temperature, flavor, texture, etc. These representations are left as future work.

Two rules are established for such transformation regulatory mandate: (1) all basic component types cannot be added or altered to prevent creation of new basic types, and (2) other component types can be freely disposed of, altered, redefined, or added. The first rule preserves the integrity of the entire 'life form' to keep the basic construct simple and unchanged. Imagine if the DNA kept evolving and breeding new types of nucleotides other than ATCG, all human races on earth would flourish beyond anyone could imagine. The second rule supports cell growth or die. So should file be created and subsequently combined to form a BASS component as depicted earlier in Figure 2 and eventually be disposed of. Details on how this structural process could be realized will be discussed in Section 4 Figure 4.

3.2. Functional aspect of system architecture

The most important functions of the proposed architecture are the global clock and type derivation. The global clock regulates all component activities and establishes the limiting TTL to prevent their execution from indefinite holding (of resource) or starvation (waiting for resource), whichever situation applies. This is analogous to the heart that generates pulses to regulate all the body functions until we die.

Building on hardware gates and software bits, type classification will take the fundamental operations associated with each type as summarized in Table 1.

Each software component consists of a type and its associated operations (or methods) to function as a unified entity. Thus, an integer component can operate with another integer component directly via ADD/SUB operations. Operations of composite and U-defined types can be constructed from basic types and combined to form complex operations. For example, a composite component built from *int* and *binary* types could define an additional multiplication operation using Booth Algorithm by combining ADD/SUB and SHT. These add-on constructs can be summarized in enhanced characteristics of Proposition 2 that will be established subsequently in Note 1 and 2.

3.3. Behavioral aspect of system architecture

An important behavioral dynamicity is the software component life cycle. Every component must undergo the three life cycle stages which are governed by a predefined TTL threshold that limits the life span of the component. Conventional systems will perform a 'context switch' to preserve all process states and contents before swapping this expiring process out. BASS performs component reproduction (creation) of new components to handle whatever functions are required to continue processing (sustainment) while the TTL of the old component diminishes. The expiring component is then disposed of (cessation). No context switch, storage reclamation, and house-keeping tasks are required. This makes the earlier fixed-size structural mandate ideal for *replacement in situ*. This is similar to old skin cells coming off as they are replaced by the new ones. Consequently, minimal overhead is involved in the system dynamicity.

One arguable issue when taking structural and functional considerations into account is the way components grow in the same manner as cells do. As DNA induces run-on strings of A, T, C, and G, identification or matching is tedious and time-consuming. Consequently, the DNA strands can be excessively long which somehow offset their uniqueness of identification advantage. On the contrary, identification of component and type under BASS scheme is confined to the structural architecture: fixed size, finite number of components, and well-defined methods. Thus, processing is fast and enumerable to terminate.

The second proposition conveys one very important architectural design principle, that is, component autonomy. It implies that each software component is self-contained. Yet its behavioral capability depends on how it is designated to function. For example, graphic components will be equipped with basic type operations, such as A, B, and C to support the sight sensory. On the other hand, communication components will contain protocol-related operations encompassing E, G, H, and S to support different operations. Since each software component must operate to practically 'survive' on its own, loose integration (coupling) is the basis for the design principle. They essentially have virtually no relation except a direct reference that ties them to the software system building blocks. Such independent constructs make homo-function software component interchangeable. By the same token, related or hetero-function software components interoperate to exchange or share results required by the sub-system to which they belong. This can be summarized in two enhanced constructional and operational characteristics of software components.

Note 1: All components can be constructed to make them interchangeable and interoperable among their shared components.

Note 2: Software engineering components can be composed in such a way that they comply with the underlying architecture and operations.

Based on the establishment of the above three architectural constructs, some distinctions to object-oriented paradigm are summarized in Table 2.

These distinctions refute the similarity to object-oriented paradigm that does not support the creation, sustainment, and cessation of any autonomous artifacts on their own account. Existing programming languages and their environments encompass heavy

Table 2. Summary of BASS vs objected-oriented architectural differences.

BASS (X)	O-O paradigm (Y)	Remark (denoted by X and Y for brevity)
Type	Attribute	X is confined to fixed hierarchy, but Y must obey the language construct
Component	Class	modular in X, but Y depends on programmer
File	Object	instance: expiry (X), explicit deletion (Y)
–	Sub-class	X reproduces component, Y uses language support
Group of components	Nested classes	X combines components, Y combines classes
Creation, sustainment, cessation	Abstraction	Behavioral aspect of X, disciplined design of Y
Simplicity	Complexity	goal of X, Y depends on programmer
–	Inheritance, polymorphism	X is autonomous, Y uses language support
Component	Encapsulation, modularity	X is mandatory, Y depends on programmer
Component	Information hiding	X is mandatory, Y is done by programmer
Reproduction	Instantiation	X splits, Y creates object from class
–	Association, overloading	X is autonomous, Y uses language construct
–	Concurrency	X is autonomous, Y uses language construct
–	IDE	X is self-contained, Y requires its support
TTL	Persistence	X is self-limiting, Y uses language construct

overhead such as compiler, libraries, application program interfaces (APIs), and integrated development environment (IDE). This makes them unsuitable to BASS design principles.

3.4. Verification of proposition 1

A corroboration of formal verification can be demonstrated as follows. Let $e \in E$, $g \in G$, $h \in H$, and $s \in S$ denote basic, composite, U-defined, and special/reserved component types and their corresponding sets, respectively. The relation $\rightarrow$ denotes *composed of*, i.e. $n \rightarrow e|g|h|s|\{g, h, s\}$ designates a component n to be composed of either pure e, g, h, s, or mixture of g, h, s, where {..} denotes mixture of types. By the component type hierarchy of Figure 3, $\forall g, h, s$, $g = \bigcup_{i=1}^{x} e_i$, $h = \bigcup_{i=1}^{y} e_i$, and $s = \bigcup_{i=1}^{z} e_i$, where x, y, z are arbitrary typing counts such that for any component n, the architectural construct must satisfy the first condition (i)* plus one of the following conditions:

(i)*	$n \cap e \neq \emptyset$		/* basic types	*/
(ii)	$n \cap g = \emptyset$	or $n \cap g \neq \emptyset$	/* composite	*/
(iii)	$n \cap h = \emptyset$	or $n \cap h \neq \emptyset$	/* U-defined	*/
(iv)	$n \cap s = \emptyset$	or $n \cap s \neq \emptyset$	/* special/reserved	*/
(I)	$n \cap (g \cap h) = \emptyset$	or $n \cap (g \cap h) \neq \emptyset$	/* composite/U-def	*/
(II)	$n \cap (g \cap s) = \emptyset$	or $n \cap (g \cap s) \neq \emptyset$	/* composite/reserved	*/
(III)	$n \cap (h \cap s) = \emptyset$	or $n \cap (h \cap s) \neq \emptyset$	/* U-def/reserved	*/
(IV)	$n \cap (g \cap h \cap s) = \emptyset$	or $n \cap (g \cap h \cap s) \neq \emptyset$	/* compo/U-def/reserved	*/

The first condition (i)* is mandatory for all components constructed since they must comply with the proposed scheme. This is in concert with the A, T, C, and G nucleotides that make up the DNA. One of the remaining conditions could hold if the component so constructed is made up of additional types. That is to say, conditions (ii) to (iv) represent pure composite, U-defined, or special/reserved type, while conditions (I) to (IV) represent composite/U-defined type, composite/reserved type, U-defined/reserved type, and composite/U-defined/reserved type, respectively. For example, define a composite component type $(n_1) \rightarrow (A, C)$ and a user-defined component type $(n_2) \rightarrow$ (extended-int, B, C). In this case, the counts of n_1 and n_2 become $x = 2$ and $y = 3$, respectively, and $n_1 \cap n_2 \neq \emptyset$ which is equal to char (C). Hence, the component having $n_1 \cap n_2$ as its typing basis satisfies conditions (i)* and (I). On the other hand, if $n_3 \rightarrow$ (extended-int, B, D), then $n_1 \cap n_3 = \emptyset$. This means that the component having $n_1 \cap n_3$ as its typing basis also satisfies conditions (i)* and (I).

Now suppose a new component type (m) is created by combining int (A), binary (B), and (n_1 and n_2), or $m \rightarrow (A, B, n_1, n_2)$ as a special type having the count $z = 4$, there are two possible formulation conditions to be deployed:

- condition (i)* and (I), i.e. consider A and B separately satisfying (i)* and the n_1 and n_2 satisfying (I), that is, $m \cap A \cap B \cap (n_1 \cap n_2) \neq \emptyset$, or
- condition (ii) and (I), i.e. consider A and B as a composite type $(A \cup B)$ satisfying (ii) which implicitly satisfies (i)*, and the entire group satisfies (IV), that is, $m \cap (A \cup B) \cap (n_1 \cap n_2) \neq \emptyset$.

It can be inferred from the above formulation that the simple building blocks make up larger components under the proposed scheme.

3.5. Verification of proposition 2 and its notes

Let l denote the height of the component type hierarchy measured from basic type which serves as the root level. Define $l = 0$ for the basic types. Since all component types are derived directly from the basic types which is one level away, the height becomes $\max(l) = 1$.

As for Note 1 goes, given p is a newly created component. There are two scenarios to consider. Firstly, $p \in G|H|S$, i.e. p is either a composite, U-defined, or special/reserved type. According to Proposition 1, the condition (i)* must hold for all components, i.e. the portion of p that is made up of basic types is known by all other components. The remaining portion that is not pure basic types must either be pure composite, U-defined, special/reserved type, or mixture of composite/U-defined, reserved/composite, reserved/U-defined, reserved/composite/U-defined. In either case, the conditions (ii) to (iv) or (I) to (IV) apply. Thus, data exchange and interaction among them are straightforward. Secondly, $p \notin G|H|S$. Only the basic portion applies to p. This implies that the remaining portion of p does not reuse any existing types and methods and is foreign to other components. In order for p to exchange or interact with others, p must deposit its type and corresponding method to the shared component repository. As a consequent, Note 1 holds.

Proof of Note 2 also follows in a similar manner. From Proposition 1, all software components must be created having at least condition (i)*, and so is the underlying architecture. Otherwise, Proposition 1 will not hold and the system cannot operate. Thus, the smallest composition of software artifact complies with the proofs and is able to operate properly. As these software components grow, the newly added constituents of type {G, H, S}, as well as their corresponding methods, must be declared and deposited in the shared component repository. At which point, each added artifact can be accessed and interoperated with other existing components. Consequently, Note 2 holds.

3.6. Potential benefits to real applications

Conventional software systems from the highest level of abstraction such as architecture, modeling, software system design, computing representations, and execution, are all transformed to 0 and 1 to run on hardware. BASS is no different but takes the biological-like

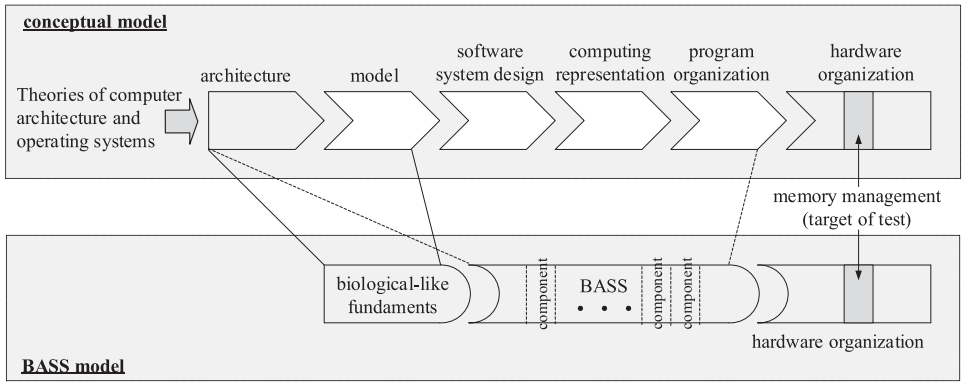

Figure 5. A mix of architectural constructs to hardware realization of BASS.

approach to synthesize the component construction, operation, and disposal processes. A suitable snapshot of testing these novelty mixes is to measure the concrete existence of components, i.e. memory allocation, deallocation, and process execution of components. Figure 5 shows the transformation from conceptual model to BASS model whose image of execution resides and transforms in memory. The reasons are two folds. First, it is a concrete evident to objectively measure BASS performance in software systems; and second, the framework can be exercised in real use to mitigate the infamous 'memory wall' problem. Under H1-H4, the memory test scheme will not make use of NUMA to complicate the access and retrieval processing since there is no shared resource to begin with.

As part of the on-going investigations, the simulation results [20] have shown that TTL-FIFO and replacement *in situ* schemes saved 21% cost over the Least Recently Used (LRU) built-in algorithm of the memory chipset. Consequently, the novelty of BASS could, at present, be empirically gauged by how well it manages the memory in comparison with the state-of-the-practice memory management methods.

Another area of application that increasingly gains research interest is computing structure. Many data gathering techniques are developed to handle massive data storage and computations such as data warehouse and big data. Rack scale architecture [18], co-locating data storage and processing [21], computing virtualization [22], scalable synchronization on shared memory multiprocessor [23], and transient servers [24], are a few prominent examples. Unfortunately, these classical paradigms rest on stored program architecture that requires increasing memory, computing power, and complex algorithms that eventually create the memory wall. BASS, on the contrary, offers a pool of short-lived resources to handle data processing in a similar manner as the human analogy. Figure 6 depicts the conceptual framework of BASS data pipeline that echoes the above multimedia processing argument. For example, receptor1 could denote the eyes to receive image (signal1), while receptor2 could denote the ears to receive sound (signal2), and so on. This in turn is processed and filtered by the respective methods and filters to extract useful information from the brain archive to consolidate and form the desired output.

The important issues for the filtering process are two folds, *coverage* and *tracking*. Coverage ensures that the system could provide sufficient receptors to accommodate different input signals. Tracking, on the other hand, must ensure that the input data are recognizable or failed otherwise. For instance, a Latin voice signal might be handled by the coverage of receptor2. However, tracking could not retrieve related information from the brain (as the person did not learn Latin) to recognize it and thus failed to understand what the input voice meant.

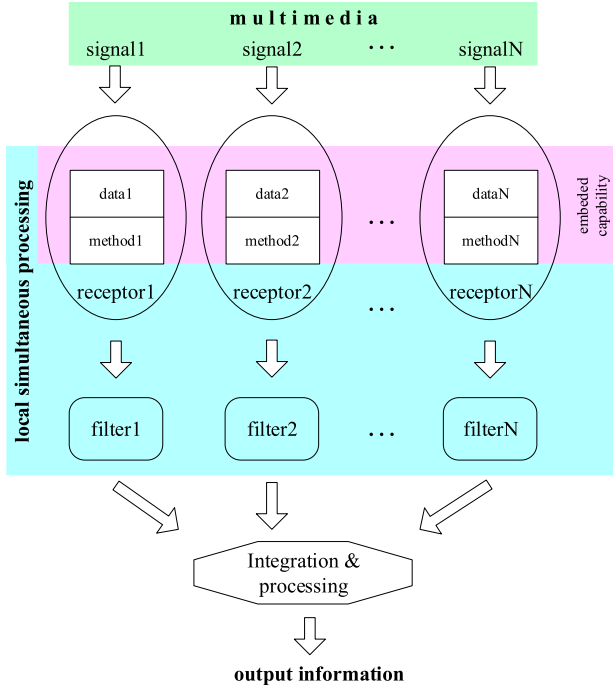

Figure 6. Data pipeline: signal reception by specific input receptors.

The last example reiterates the compatibility with both propositions, namely, (1) no typing mismatch since the underlying architecture separates receptors for different input types and operations to be processed accordingly, (2) retrieval without conversion since each receptor is built to handle a specific input type, (3) simple building blocks and direct reference using simple state-machine having as few states as possible, and (4) simultaneous integration and processing as all inputs are processed concurrently to yield an immediate outcome.

4. Experiment

A simulation module was devised to try out the above test scheme. Design of the component was carried out as a fixed size linear array in accordance with the cell structure, i.e. from nucleotide to codon to chromosome and cell. Each hierarchy represented the characteristics of basic building blocks so stated that made up a BASS component. The overview of a software component is depicted in Figure 7. Notice that the size field holds the configurable component size that allows system administrator to tailor the implementation to suit the running

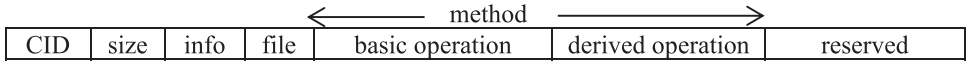

Figure 7. Overview of a BASS component.

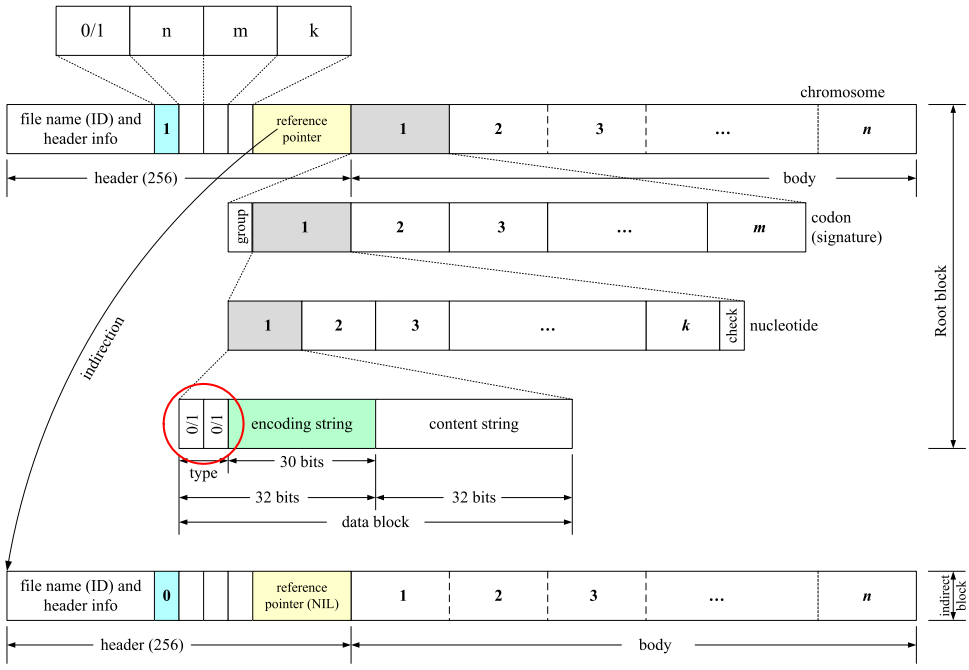

Figure 8. Structure of a file.

platform. Thus, infinite enumeration will not be the case as in the DNA strands.

A component consists of a unique component ID or *CID*. The *size* field holds the number of bytes occupied. The *info* field contains basic data such as date of creation, last modified, TTL timestamp, etc. The *file* field will be elucidated in the next Figure. The *method* field contains basic operations needed to run the basic type (E) and derived operation to run user-defined type (G|H|S). The *reserved* field, as the name implies, is for future use.

The important part of a software component is *file* which is described in Figure 8. The first block is a directly accessible *root* block to permit hardware implementable access and retrieval operations. A file consists of two parts, namely, header and body. The header of size 256 bytes contains file name or ID and header information such as file location, owner, date of creation/modification. The next four values are set as follows:

- indirection bit (0/1), where 0 denotes direct, 1 denotes indirect reference of additional file blocks;
- multiplicity (n), whose values are either 1 (default value) or 2 … n. These blocks analogously denote the chromosomes;
- system addressable size (m), whose value is divisible by *group*. This block size analogously denotes the codon. The value of group designates the number of nucleotides that constitutes units of file. The default value is 3 to be in-line with natural codon; and
- number of data blocks per process (k), whose value is power of 2, the default value is 4 blocks to mimic the natural A, T, C, G code. A *check* byte is appended at the end of each block as the sentinel value for its signature. These blocks analogously denote the nucleotides.

The values of n, m, and k are configurable to fit the operating platform set up. Since data are stored in the nucleotide block, the smallest possible size takes $k = 1$. This in turn presets the values of $m = 1$ and $n = 1$ which render the component to mimic a uni-cellular life form. The largest possible size depends on the limitations of the implementation platform which could mimic a large organ.

The last value of header block is a reference pointer, which is set according to the indirection bit. If the indirection bit is 1, the reference pointer will hold the address of the next file block which is structurally identical to the root block. Otherwise, indirection bit is 0 and the reference pointer is set to NIL.

The body holds n blocks of file contents. Each block or chromosome further holds m blocks of codon having a prefix group byte. The group byte designates the number of codons that constitutes a unique type representation. Each codon in turn holds k blocks of nucleotide and a check byte. The breakdown of each nucleotide encompasses a 64-bit block, hereafter will be referred to as *data* block, that is divided into two equal halves. The first half stored the component types defined in Figure 3. The first two bits identify the component types as follows:

- 00 – basic types;
- 01 – composite type;
- 10 – user-defined type; and
- 11 – special/reserved type.

The remaining 30 bits of the first half contain the bit pattern of the encoding string that defines how non-basic types are encoded, i.e. all zeros denote basic type, all ones denote special/reserved type, and other patterns denote a specific composite or user-defined types.

The second 32-bit half contains the content string of the component which is defined by a standard encoding scheme. The proposed construct of content string could be implemented as a hardware executable representation to be pursued in future work.

Table 3. Simulation parameter setup.

Parameter	Setup value	Remark
Mean lifetime (τ)	500	475 rounded up
Initial component (N_0)	1	First component
Simulation run (t)	2000	Round up
Processing repetition (N)	10,000	Arbitrary selected

There are two reasons behind the design philosophy of file: (1) to preserve the correspondence with natural cells, and (2) to make provision for close imitation with organs. As all current file structures are ASCII and binary to represent sight and sound, these formats apparently cannot cope with the remaining three sensory artifacts that are currently unexplored.

The simulation procedure was laid out as follows.

```
Procedure
    create FIRST component
    set LIMIT
    X ß FIRST, r ß 0, w ß 0
  reproduce:
    repeat
def_op (newM X')
create CID' from CID + timestamp
init_defaults
process_component (X')
assign_component (X')
run_service()
TTL_check()
    until w > LIMIT
end procedure
```

The simulation process randomized creation of components except the first one. Table 3 summarizes the estimation of parameter setup. Some parameters deployed in the simulation were experimentally tried and predicted using exponential decay (Eq 1) and the 37% Method [25] to simulate cell reproduction process, that is,

$$N(t) = N_0\, e^{(-t/\tau)} \tag{1}$$

where $\tau = 1/\lambda$, λ denotes the creation rate.

The proposed design was written in C running on Intel® Core i5, CPU M 520 @2.1 GHz x 4, Ubuntu 16.04 LTS system to simulate BASS memory allocation scheme. The execution performed basic arithmetic and modulo computations of 10,000 repetitions. Components were created, sustained (executed), and disposed of (written to disk) very rapidly. For the first hundred repetitions, memory allocation calls in C were performed repeatedly in user mode. This inevitably induced high overhead to set up the component and file structures which sporadically took several clock ticks during each simulation run, i.e. 3, 5, 10, 12, 13, 17, 18, 20. An extended looping of 500,000 repetitions was then conducted in each run to observe whether any steady states would result. It could be seen that more repetitions induced the OS to prolong the process in core so that execution would be completed faster. In other words, memory blocks were replaced *in situ* and fewer running processes being swapped out, hence faster process completion. Notice that the tail flattens down considerably. This phenomenon can be observed in Figure 9. The total execution time of subsequent calls was negligible. The results are shown in Tables 4–5 and Figure 9.

```
sim_test
/* handcraft the FIRST component */
/* set no. of simulation runs */

/* run component until LIMIT */

/* create new component */

/* all defaults initialization */
/* assemble component */
/* deposit in FIFO memory */
/* computation processing */
/* expiration */
```

Time measurements were done by system clock ticks via *struct tms*. The variables duration denotes the time of simulation run (sim_test), DFutime denotes component cycle time (steps 1–7), and DFstime denotes the time spent on instructions of the user calling process (memory allocation calls in C).

Table 4. Simulation results of 10,000 times on 21 runs.

Process start	Process stop	Duration	STutime	ENutime	DFutime	STstime	ENstime	DFstime
1718587188	1718587188	0	0	0	0	0	0	0
1718587188	1718587190	2	0	0	0	0	1	1
1718587190	1718587216	26	0	8	8	1	17	16
1718587216	1718587220	4	8	10	2	17	19	2
1718587220	1718587230	10	10	13	3	19	25	6
1718587230	1718587231	1	13	14	1	25	26	1
1718587231	1718587232	1	14	14	0	26	26	0
1718587232	1718587234	2	14	15	1	26	27	1
1718587234	1718587235	1	15	16	1	27	28	1
1718587235	1718587239	4	16	18	2	28	30	2
1718587239	1718587243	4	18	19	1	30	32	2
1718587243	1718587247	4	19	21	2	32	35	3
1718587247	1718587251	4	21	23	2	35	37	2
1718587251	1718587255	4	23	24	1	37	39	2
1718587255	1718587259	4	24	26	2	39	42	3
1718587259	1718587263	4	26	28	2	42	44	2
1718587263	1718587267	4	28	29	1	44	46	2
1718587267	1718587271	4	29	31	2	46	49	3
1718587271	1718587275	4	31	32	1	49	51	2
1718587275	1718587279	4	32	34	2	51	54	3
1718587279	1718587283	4	34	35	1	54	56	2

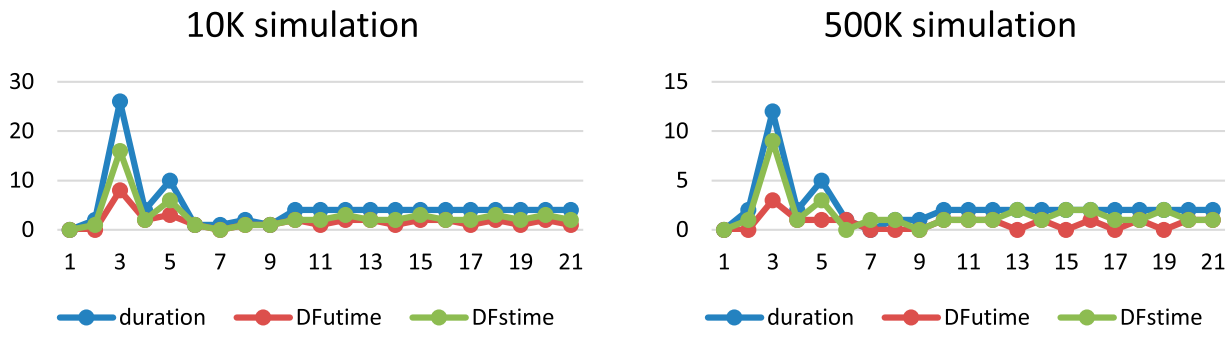

Figure 9. Memory allocation looping of 10k and 500k after 21 runs.

Table 5. Simulation results of 500,000 times on 21 runs.

Start	Stop	Duration	STutime	ENutime	DFutime	STstime	ENstime	DFstime
1718591454	1718591454	0	0	0	0	0	0	0
1718591454	1718591456	2	0	0	0	0	1	1
1718591456	1718591468	12	0	3	3	1	10	9
1718591468	1718591470	2	3	4	1	10	11	1
1718591470	1718591475	5	4	5	1	11	14	3
1718591475	1718591476	1	5	6	1	14	14	0
1718591476	1718591476	0	6	6	0	14	15	1
1718591476	1718591477	1	6	6	0	15	16	1
1718591477	1718591478	1	6	6	0	16	16	0
1718591478	1718591480	2	6	7	1	16	17	1
1718591480	1718591482	2	7	8	1	17	18	1
1718591482	1718591484	2	8	9	1	18	19	1
1718591484	1718591486	2	9	9	0	19	21	2
1718591486	1718591488	2	9	10	1	21	22	1
1718591488	1718591490	2	10	10	0	22	24	2
1718591490	1718591492	2	10	11	1	24	26	2
1718591492	1718591494	2	11	11	0	26	27	1
1718591494	1718591496	2	11	12	1	27	28	1
1718591496	1718591498	2	12	12	0	28	30	2
1718591498	1718591500	2	12	13	1	30	31	1
1718591500	1718591502	2	13	14	1	31	32	1

The proposed BASS architecture establishes an unconventional basis for building software systems. The design framework mimics the uni-cellular life form to arrive at the simple structures (basic, composite, user defined, and special typing) that are linearly arranged to permit straightforward functions (basic ADD, SUB, . . . , FRQ) and behaviors (TTL-FIFO and replacement *in situ*), thereby supporting the four characteristics of software architecture and operating environments.

It was apparent that BASS design principles based on H1-H7 made it easy for hardware realization. The above simulation and the simulation results obtained by [20] reaffirmed the technology transfer, to some extent, at memory level (Q1). The importance of H7 mandated exclusion of logical level complexity such as NUMA, DNA and quantum computing. These architectural constructs could not be mapped directly to simple gate-level implementation (Q3). On the contrary, evidence could be observed that the fixed-size linear configuration of BASS components would structurally map to hardware implementation and functionally run the TTL-FIFO in the memory chipset with replacement *in situ*. Consequently, little software overhead for memory access and retrieval was involved, which in turn would benefit the speed gain (Q2).

The very fact of H7, in conjunction with what was exemplified in Figure 7, made it viable to create unique, simple, but effectively workable components to handle specific processing that incurred no typing mismatch, retrieval conversion, or complex theoretical imposition (Q3).

The benefits advocate new system construction by several folds:

(1) Fast access and retrieval to component configuration since its structure is fixed size and arranged linearly. This setup makes it ideal for table lookup or base register and offsetting implementation at hardware level.
(2) Components are governed by predetermined TTL. Thus, all components come and go. There is virtually no burden left over for garbage collection.
(3) Components are self-contained and operationally autonomous. It is thus portable to different support environments.
(4) The need for large, preset, and long-term availability of resources such as storage space or processing units are unnecessary. By virtue of the first and second benefits above, these resource requirements can be established on demand and released once the execution is over. Thus, energy conservation can be substantial.
(5) Lessen the burden of release engineering as new components can be tailored to fit a specific locale or 'ecosystem' as Rossi puts it [26] without having to maintain continuous delivery across the board. This flexibility is similar to organ transplant that is performed on a case-by-case basis as deemed necessary.

5. Conclusion

The biological-like architecture for software systems (BASS) are proposed to revolutionize how software systems should be designed. By imitating cell life cycle, the proposed architecture regulates components that run, split, and deallocated as natural as they can be.

The prospectus is justified by three architectural constructs, namely, structural, functional, and behavioral. Two propositions are precipitated to govern the architecture of BASS. The most fruitful result of BASS is an autonomous entity that is self-reliance and independent among themselves.

Therefore, software components under BASS can grow and be disposed of with little overhead in the same way as a man is born and passed away. This very notion will maintain the dynamicity and independence of computer applications in the same manner as the human society, but less congestion since every new application generation will go through this three-stage life cycle indefinitely for the existence of the computer systems.

Future creation work of BASS are design and implementation of components for the remaining three senses to complete the journey. An important consideration is the design quality of BASS components, in particular, Measure of Aggregation (MoA) [27]. This metric could be deployed to measure how composite and U-def types will permit flexibility and effectiveness of attributes to support component autonomy.

Finally, one of the important problems is autonomous method execution. As researchers still cannot uncover the secrets of DNA that would shed lights on these issues, how different methods of a designated artifact can run in different environments where it resides still remains to be reckoned with. The hardware solution at cache level where the basic types *live* is under investigation. This viable solution will hopefully bring about BASS to a new level of ubiquitous architecture for any operating environments.

Disclosure statement

No potential conflict of interest was reported by the author(s).

References

[1] Fowler M. Patterns of enterprise application architecture. Boston, USA: Addison-Wesley; 2003.

[2] Gershenfeld N, Chuang I. Quantum computing with molecules. Scientific American; 1998. p. 66–71, 278, 6, USA.

[3] Braich R, Chelyapov N, Johnson C, et al. Solution of a 20-variable 3-SAT problem on a DNA computer. Science. 2002;296:499–502. doi:10.1126/science.1069528

[4] Adleman L. Computing with DNA. CMU, Pittsburgh, PA, USA: Scientific American; 1998. p. 54–61.

[5] Mayr E. The growth of biological thought—diversity, evolution, and inheritance. Cambridge, MA: The Belknap Press of Harvard University Press; 1982.

[6] Foster M, Sharp R. Race, ethnicity, and genomics: social classifications as proxies of biological heterogeneity. Genome Res. 2002;12:844–850.

[7] Parnas D. On the criteria to be used in decomposing systems into modules. Commun ACM. 1972;15(12):1053–1058. doi:10.1145/361598.361623

[8] Meierk JD, Hill D, Homer A, et al. Microsoft application architecture guide, patterns & practices, 2nd ed. Redmond, WA, USA: Microsoft Press; 2009.

[9] Reference model for framework of software engineering environments. Edition 3 of Technical Report ECMA TR/55, NIST Special Publication 500-211, 1993.

[10] Abraham BT, Aguilar JL. Middleware for improving performance in a component-based software architecture. Int J Comput Appl. 2012;34(1): 25–28.

[11] Xia S, You J. A coordination model for network computing environments. Int J Comput Appl. 2002;24(3):153–159. doi:10.1080/1206212X.2002.11441675

[12] Saulsbury A, Pong F, Nowatzyk A. (1996). Missing the memory wall: the case for processor/memory integration, Proceedings of the 23rd Annual International Symposium on Computer Architecture (1996), Philadelphia, USA, May 22–24, 1996, p. 90–101.

[13] Rixner S, Dally W, Kapasi U, et al. Memory access scheduling. Proceedings of the 27th Annual International Symposium on Computer Architecture; 2000. Vancouver, Canada, p. 128–138.

[14] Blagodurov S, Fedorova A, Zhuravlev S, et al. A case for NUMA-aware contention management on multicore systems. Proceedings of the 19th International Conference on Parallel Architectures and Compilation Techniques; 2010, Vienna, Austria.

[15] Lergchinnaboot G, Sophatsathit P. A biological-like memory allocation scheme using simulation. Proceedings of the 2nd International Conferences on Information Technology on Information Technology, Information Systems and Electrical Engineering; 2017. Yogyakarta, Indonesia, p. 425–428.

[16] Watada J, abu Bakar Rb. DNA computing and its applications. Eighth International Conference on Intelligent Systems Design and Applications; 2008, p. 288–294. doi:10.1109/ISDA.2008.362

[17] Adleman L. Molecular computation of solutions to combinatorial problems, science. New Series. 1994;266(5187):1021–1024.

[18] Costa P, Ballani H, Razavi K, et al. R2C2: a network stack for rack-scale computers. SIGCOMM '15, August 17–21, 2015, London, p. 551–564.

[19] Cockroaches survive nuclear explosion, mythbusters database. http://dsc.discovery.com/tv-shows/mythbusters/mythbusters-database/cockroaches-survive-nuclear-explosion.htm, accessed on October 29, 2018.

[20] Lergchinnaboot G, Sophatsathit P, Maneeroj S. In situ caching using combined TTL-FIFO algorithm. 2019 IEEE 5th International Conference on Computer and Communications; 2019. Chengdu, People's Republic of China, p. 437–441.

[21] O'Driscoll A, Daugelaite J, Sleator R. 'Big data', hadoop and cloud computing in genomics. J Biomed Inform. 2013;46:774–781. doi:10.1016/j.jbi.2013.07.001

[22] Heitzler M, Hackl J, Adey B, et al. A method to visualize the evolution of multiple interacting spatial systems. J. Photogramm Remot Sens. 2016;117:217–226. doi:10.1016/j.isprsjprs.2016.03.002

[23] Mellor_Crummey J, Scotty M. Algorithms for scalable synchronization on shared-memory multiprocessors. ACM Trans Comput Syst. 1991;9(1):21–65. doi:10.1145/103727.103729

[24] Singh R, Sharma P, Irwin D, et al. Here today, gone tomorrow: exploiting transient servers in datacenters. IEEE Internet Comput. 2014: 22–29. doi:10.1109/MIC.2014.40

[25] Breiman L. Randomizing outputs to increase prediction accuracy in machine learning. Machine Learning. 2000;40:229–242.

[26] Adams B, Bellomo S, Bird C, et al. The practice and future of release engineering: a roundtable with three release engineers. IEEE Softw. 2015;32(2):42–49. doi:10.1109/MS.2015.52.

[27] Bansiya J, Davis C. A hierarchical model for object-oriented design quality assessment. IEEE Trans Software Eng. 2002;28(1):4–17. doi:10.1109/32.979986